



CSE 406 — Cyber Security Sessional
Project 2026: Attack Tools Implementation

TCP Reset Attack on Video Streaming

Project Design Proposal

Submitted by

Group No: 03, Section : A1

Student ID	Name
2105004	Fahim Ishtiak
2105001	Nahid Hossain Redom

Date of Submission: September 11, 2026

Abstract

This document presents the design proposal for implementing a **TCP Reset (RST) Injection Attack** targeting an active video streaming session. TCP was not designed with authentication for its control segments; consequently, any adversary who can observe or correctly guess the four-tuple (source IP, destination IP, source port, destination port) and an in-window sequence number of an established connection can forge a segment with the RST flag set and force premature termination of that connection. We propose a two-stage implementation: an initial single-host simulation using Docker containers connected via a user-defined bridge network, followed by a physical three-machine demonstration (video server, client, and attacker on separate hosts) to validate the attack under realistic network conditions. Because both a Docker bridge and a physical LAN switch forward unicast frames only to their destination, our attacker first uses **ARP cache poisoning** to place itself on-path as a man-in-the-middle, from which it observes the exact live sequence numbers and injects the forged RST. This report details the network topology, the relevant packet structures, a phased implementation plan, the expected outcome of a successful attack, and candidate defense mechanisms, most notably the sequence-number validation rules of RFC 5961 and ARP-layer protections such as Dynamic ARP Inspection.

1 Overview of the Project Idea

The Transmission Control Protocol (TCP) provides reliable, ordered, connection-oriented communication, but it was designed without any built-in mechanism to authenticate its control-plane segments. A TCP connection is uniquely identified only by a 4-tuple — (source IP, source port, destination IP, destination port) — together with a 32-bit sequence number space used to order data and validate incoming segments. If an attacker can construct a single spoofed segment carrying the RST (Reset) flag, the correct 4-tuple, and a sequence number that the receiver's TCP stack considers acceptable, the receiver will tear down the connection immediately, as though the legitimate peer had requested termination.

1.1 Why Video Streaming is an Interesting Target

Video streaming over TCP (e.g. progressive HTTP download, an RTSP stream with interleaved TCP transport, or a custom TCP stream) typically involves a single *long-lived* TCP connection carrying a continuous, high-volume data flow. This makes it an instructive target for three reasons:

- **Observability window:** a long-lived connection gives an on-path attacker ample time to sniff traffic and extract the parameters needed to forge a valid RST segment.
- **Visible, measurable impact:** unlike an isolated dropped packet (which adaptive bitrate logic and client-side buffering typically mask), a full connection reset produces an abrupt and observable disruption — playback stalls, the player reports a connection/network error, and the socket disappears from both peers' connection tables.
- **Real-world relevance:** forged RST injection is not merely academic — it is the mechanism historically used by some network-level censorship and traffic-management systems to unilaterally terminate connections they wish to block. Our project reproduces this technique at a small, controlled scale.

1.2 Choice of Streaming Protocol (and What to Avoid)

The reliability of the demonstration depends heavily on choosing a transport that maps the whole video session onto *one* long-lived TCP connection. We therefore plan to use one of: (a) a **custom raw-TCP video streamer** (a small server that pushes a .ts/.mp4 file over a single socket to a matching client) for maximum control and reproducibility, or (b) **RTSP with interleaved (TCP) transport** using a standard server/player. Both keep a single victim connection open for the duration of playback, which is exactly what a single injected RST tears down.

We explicitly avoid two categories that would muddy the results:

- **QUIC / HTTP-3:** these run over UDP, which has no connection state and no RST flag, so a TCP reset attack simply does not apply. Many modern browsers prefer QUIC for video, so a naive “point the browser at a public site” demo may silently use a transport immune to the attack.
- **Adaptive HLS/DASH over many short HTTP connections:** these fetch video in many short-lived segment requests; resetting any one of them causes at most a momentary stall before the player opens the next connection, making a clean, repeatable single-reset demonstration difficult. We treat client auto-reconnection as a result to *measure* (Section 5), not as our primary target transport.

1.3 Threat Model and Scope

We explicitly scope the project to an **on-path (LAN-local) attacker**: a host that resides on the same network segment as the client and that makes itself on-path via **ARP cache poisoning**, so that it can *directly relay and sniff* live TCP traffic and read the exact sequence number in use, rather than needing to guess it blindly. This is the standard, tractable scope for a course-level implementation. A true **off-path (blind) attacker** would additionally need to defeat source-port randomization and brute-force a valid sequence number, which is a substantially harder and lower-probability attack (discussed in Section 6 in the context of RFC 4953/5961). Our implementation and defense discussion are both built around the on-path model.

Ethical use and containment. All experiments will be conducted exclusively within an isolated Docker bridge network and, in the second phase, within a physically isolated LAN segment set up specifically for this project. No third-party, campus, or production traffic will ever be touched, and no ARP poisoning or RST injection will be performed against any host outside our own test bed. The tooling is developed solely to demonstrate and then *defend against* the attack for coursework.

2 Network Topology, Components, and Timing Diagram

The project is implemented in two deployment phases that share an identical logical topology: a **Video Server**, a **Client (victim)**, and an **Attacker**, all on the same broadcast domain.

2.1 Phase 1: Single-PC Simulation using Docker

In the first phase, all three roles are realized as separate Docker containers connected to a common user-defined **bridge network**, running on a single physical machine. Docker’s bridge network places all containers on the same virtual Layer-2/Layer-3 segment (a private subnet, e.g. $172.20.0.0/24$), which faithfully reproduces the on-path/LAN-local scenario without requiring physical hardware.

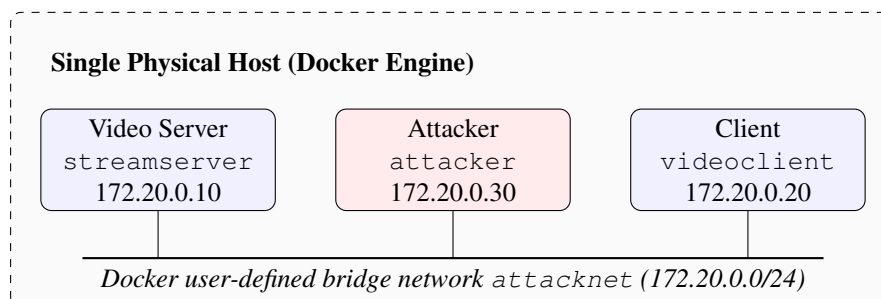


Figure 1: Phase 1 topology: three Docker containers on a single host, joined by a common bridge network. Because the Docker bridge is a learning switch, the attacker container becomes on-path via ARP poisoning (Section 2.3) rather than by passive promiscuous sniffing alone.

Components (Phase 1):

- `streamserver` — a lightweight container (e.g. based on `python:3-slim` or `nginx`) serving a video file over TCP, or running a minimal RTSP/TCP streaming server.
- `videoclient` — a container running a command-line TCP video client (e.g. `ffplay`, `curl` piping into `ffmpeg`, or a small custom Python TCP client) that fetches/streams the video from `streamserver`.
- `attacker` — a container with `NET_ADMIN` and `NET_RAW` capabilities (`-cap-add=NET_ADMIN`, `-cap-add=NET_RAW`), running Scapy/raw-socket tooling to ARP-poison, sniff, and inject spoofed segments.

2.2 Phase 2: Three-Machine Physical Demonstration

Once the attack is verified in the containerized environment, the same logical roles are mapped onto three physical machines connected to a common LAN segment (e.g. via a switch or a shared Wi-Fi access point), to validate that the attack behaves consistently outside the simplified virtual network stack.

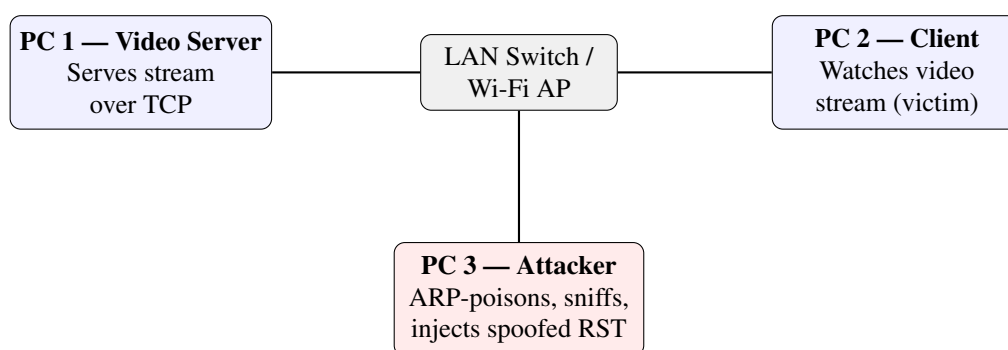


Figure 2: Phase 2 topology: three physical machines on a shared LAN segment. On a switched LAN the attacker cannot passively see the Server↔Client unicast flow; it uses ARP poisoning to relay and observe it.

2.3 Achieving On-Path Visibility (ARP Spoofing)

A crucial practical point that distinguishes a working demonstration from a non-working one: **both a physical LAN switch and a Docker bridge are learning switches**. They forward each unicast frame only out of the port associated with the destination MAC address. Consequently, an attacker in promiscuous mode sees broadcast/multicast traffic (including ARP) but *not* the unicast video segments flowing between server and client. Simply enabling promiscuous mode is therefore not sufficient to obtain the live sequence numbers.

To become genuinely on-path, the attacker uses **ARP cache poisoning**:

1. Send forged ARP replies telling the *client* that the server’s IP maps to the *attacker’s* MAC, and (symmetrically) telling the *server* that the client’s IP maps to the attacker’s MAC.
2. Enable IP forwarding on the attacker (`echo 1 > /proc/sys/net/ipv4/ip_forward`) so that relayed traffic continues to flow and the streaming session stays alive — otherwise the attacker would simply black-hole the connection, which is a denial of service but not the RST attack we are studying.
3. With traffic now transiting the attacker, read the exact live sequence/acknowledgment numbers directly off the relayed segments, then inject the forged RST at the chosen moment.

This makes the “on-path” assumption in our threat model concrete and reproducible. On a true shared medium (an old-style hub, or open Wi-Fi captured in monitor mode) passive sniffing alone would suffice; on the switched/bridged networks we actually use, ARP poisoning is the mechanism that provides

visibility. As a fallback for Phase 2, a managed switch with a **SPAN/port-mirror** port could feed the attacker a copy of the traffic instead, which we may use to cross-check results independently of ARP poisoning.

2.4 Timing / Sequence Diagram

Figure 3 illustrates the attack timeline, identical in both deployment phases at the logical level.

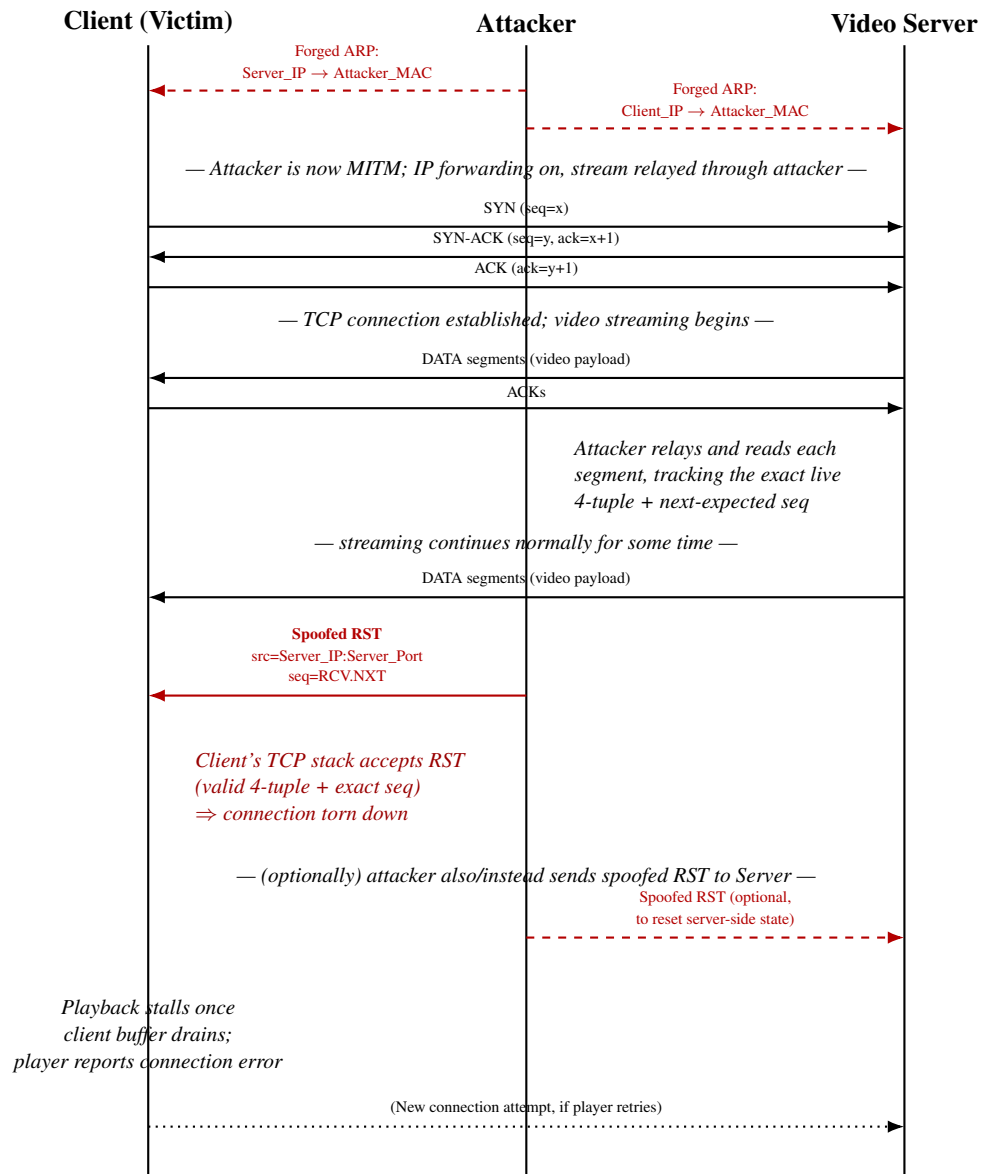


Figure 3: Timing diagram of the TCP Reset injection attack against an active video streaming connection, including the initial ARP-poisoning step that establishes the on-path (MITM) position.

3 Packet and Frame Structure

The attack requires forging a single TCP segment (encapsulated in an IP packet) with specific header fields set correctly; no application-layer payload is required. Figure 4 shows the TCP header with the fields relevant to this attack highlighted, and Table 1 explains why each is required and how its value is obtained. The preceding ARP-poisoning step forges a much simpler frame (an ARP reply binding a victim IP to the attacker’s MAC), which we mention here for completeness but do not diagram.

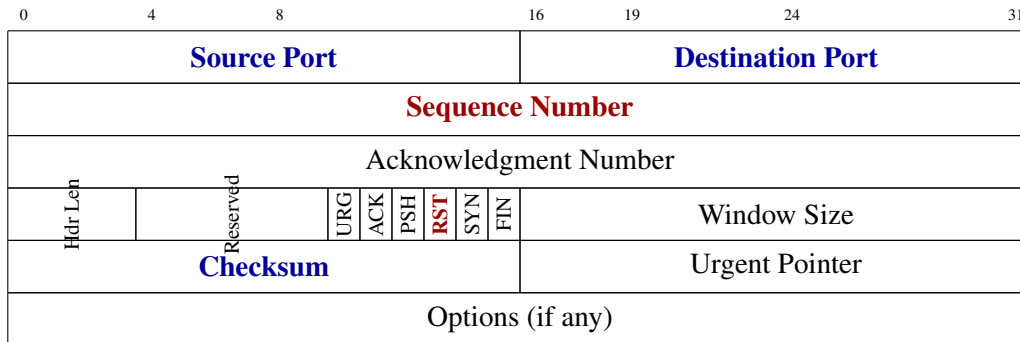


Figure 4: TCP header. Fields shown in red (Sequence Number, RST flag) are the ones the attacker must set correctly relative to the live connection state; fields in blue (ports, checksum) must exactly match the victim connection and be recomputed for the forged packet. The 20-byte IP header (source/destination IP) sits beneath this and is spoofed identically. The six 1-bit control-flag boxes (URG, ACK, PSH, RST, SYN, FIN) are rotated for legibility given their narrow width.

Field	Requirement	How obtained / rationale
Source / Destination IP	Must exactly match the real Server/Client IP pair (spoofed — attacker’s own IP is never used)	Read directly from the relayed packets while acting as MITM
Source / Destination Port	Must exactly match the live connection’s port pair	Read directly from the relayed packets
Sequence Number	Must fall within (ideally, exactly equal to) the receiver’s next-expected sequence number <code>RCV.NXT</code>	Computed as <code>seq + payload_length</code> of the most recently relayed data segment from the same direction
RST flag	Must be set to 1; other flags typically cleared	Fixed value set by the attacker’s crafting tool
Checksum	Must be a valid checksum over the pseudo-header + TCP header (+ payload, if any)	Recomputed by the crafting library (Scapy/raw sockets) — an invalid checksum causes silent drop
Payload	Not required	An RST segment carries no data; the segment is a bare header

Table 1: TCP/IP header fields relevant to constructing a valid spoofed RST segment.

Note on sequence-number strictness. Classic descriptions of this attack (pre-2010) state that any sequence number falling within the receiver’s advertised window is accepted. Modern kernels implementing **RFC 5961** are considerably stricter: a RST is accepted *immediately* only if its sequence number *exactly* matches `RCV.NXT`; an in-window but non-exact match instead triggers a “challenge ACK” rather than an immediate reset. Because our attacker is on-path and reads the exact live sequence number, it can meet this exact-match requirement — but see the timing caveat in Section 3, and the Defense discussion in Section 6.

Timing caveat under high throughput. On a fast stream, `RCV.NXT` advances continuously. Between the moment the attacker reads a sequence number and the moment its forged RST reaches the victim,

the server may already have delivered more data, so a value that was exactly `RCV.NXT` an instant ago is now stale — and under RFC 5961 a stale-but-in-window RST yields only a challenge ACK, not a reset. We therefore plan to (i) prototype in Scapy but move the hot path to C/raw sockets if injection latency is too high, and (ii) inject a small *burst* of RSTs spanning a few plausible sequence values (e.g. around `RCV.NXT` in one-MSS steps) to absorb the drift. We will report how many attempts a successful reset actually takes.

4 Implementation Plan

The implementation is organized into six phases, moving from a fully containerized simulation to a physical multi-machine demonstration and, time permitting, a working defense.

Phase A — Docker-based Environment Setup

- Define a Docker Compose file with three services (`streamserver`, `videoclient`, `attacker`) attached to a common user-defined bridge network (`attacknet`).
- Grant the `attacker` container `NET_RAW` and `NET_ADMIN` capabilities so it can open raw sockets, send forged ARP/IP packets, and enable IP forwarding; the other two containers require no special capabilities.
- Build/select a minimal video server image (a small server pushing a sample `.mp4/.ts` over a single TCP socket, or a lightweight RTSP/TCP server) and a matching client (`ffplay/ffmpeg`, or a custom Python TCP client for full control and reproducibility), per the protocol choice in Section 1.2.

Phase B — Establishing the On-Path Position (ARP Spoofing)

- Implement ARP cache poisoning of the client and server (Scapy, or a tool such as `arp spoof`), and enable IP forwarding so the stream continues to flow through the attacker.
- Verify MITM is live: confirm in Wireshark that Server↔Client segments are now visible at the attacker, and that playback is unaffected (a clean MITM, not yet a DoS).

Phase C — Traffic Capture (Sniffing)

- Implement a sniffer in the `attacker` container using Scapy's `sniff()` (or a raw `AF_PACKET` socket for performance) filtered on the Server↔Client TCP port/IP pair.
- For each observed data segment, extract source/destination IP and port, and compute the next-expected sequence number (`RCV.NXT`) for the corresponding direction.

Phase D — Spoofed RST Injection

- Implement a *sniff-and-spoof* loop: upon observing a data segment, immediately craft and send a matching forged RST toward the victim (and, optionally, a second forged RST toward the server to reset both endpoints' state).
- Prototype first in Scapy for development speed; if injection latency proves too high relative to the live sequence number advancing, reimplement the hot path in C using raw sockets, and/or use the small-burst strategy of Section 3.
- Cross-validate constructed packets against Wireshark before injecting, to catch checksum/field errors early.

Phase E — Testing and Measurement (Docker, then Physical)

- Verify the attack end-to-end inside Docker: confirm via Wireshark that the injected RST is accepted and the connection is torn down; observe the corresponding failure in the client player.
- Repeat the identical tooling on three physical machines connected via a shared switch/AP, to confirm the attack is not an artifact of the simplified Docker virtual network stack.
- Record quantitative results across repeated trials (Table 2): success rate, time from attack start to reset, and number of injection attempts required.

Phase F — Defense Prototype (time permitting)

- Demonstrate the RFC 5961 exact-match behavior on a modern Linux victim (comparing an off-path/in-window guess, which is rejected with a challenge ACK, against our on-path exact-match RST, which succeeds).
- Demonstrate an ARP-layer defense — static ARP entries or, on a managed switch, Dynamic ARP Inspection (DAI) — and show that with the MITM prevented, the on-path attacker can no longer read the live sequence number and the attack degrades to the much harder blind case.

Tooling summary: Docker & Docker Compose; Scapy (Python) and `arpspoof` for ARP poisoning; raw sockets (C, for the performance-critical RST path); `libpcap`/Wireshark (capture and verification); `ffmpeg`/`ffplay` or a custom TCP client (video); and `iptables`/`nftables`, static ARP, and DAI (defense prototypes).

5 Expected Outcome of a Successful Attack

A successful run of the attack is expected to produce the following, observable and measurable, outcomes:

- **On the wire:** a Wireshark capture on the client side showing the injected RST segment immediately following a run of normal data segments, with no preceding FIN/FIN-ACK exchange — visually distinguishable from a graceful stream end.
- **At the client:** the TCP connection entry for the stream disappears from the client’s connection table (e.g. as shown by `netstat/ss`); the video player stalls once its playback buffer is exhausted and reports a network/connection error.
- **At the server (if targeted too):** the corresponding half of the connection is likewise torn down, and the server-side process/socket handling that stream terminates or logs a reset.

Metric	How measured	What it tells us
Attack success rate (%)	Successful resets over N repeated trials	Reliability of the attack under our conditions
Time-to-disruption	From attack trigger to player stall/error	Responsiveness of the attack
RST attempts per success	Count of injected RSTs per teardown	Impact of the RFC 5961 exact-match timing drift
Behavior vs. client retry	Whether playback recovers on its own	Whether sustained injection is required
Success with vs. without defense	Rerun against RFC 5961 / DAI-hardened victim	Effectiveness of each defense

Table 2: Planned experiments and metrics to report.

Expected limitation to document, not hide. Many modern streaming clients (browsers, adaptive players) automatically attempt to re-establish a new connection after an unexpected reset, which may present

as a brief stutter rather than a permanent failure unless the attacker sustains repeated injection. We will report this behavior explicitly as part of our results rather than treating a single successful reset as the only measure of success.

6 Defense Idea

6.1 Primary Defense: Strict Sequence-Number Validation (RFC 5961)

The most direct and standardized defense against RST injection is the tightened validation rule specified in **RFC 5961**, “**Improving TCP’s Robustness to Blind In-Window Attacks.**” Instead of accepting any RST segment whose sequence number merely lies somewhere within the current receive window (the historically exploitable rule), a compliant receiver applies:

1. If the sequence number lies *outside* the current window, silently drop the segment.
2. If the sequence number *exactly* matches `RCV.NXT`, reset the connection (this is the only case that succeeds).
3. If the sequence number is *within* the window but does *not* exactly match `RCV.NXT`, do not reset; instead send a “challenge ACK” and drop the segment, forcing any genuine reset attempt to be re-sent with the correct value.

This narrows the attacker’s valid guess space from the entire receive window (which can be tens of thousands of values, or more on high-bandwidth-delay-product paths) down to a single exact value, which is precisely why our on-path attack — which reads the exact live sequence number rather than guessing — remains effective against this defense, while a blind/off-path attacker is made dramatically less likely to succeed. We plan to demonstrate this contrast directly: the defense meaningfully raises the bar for an off-path attacker, but is not, by itself, a complete defense against an attacker who can observe the connection.

6.2 Complementary Defense: Preventing the On-Path Position (ARP-Layer)

Because our attack depends on ARP poisoning to become on-path in the first place, defenses at the ARP layer are especially relevant to our threat model:

- **Dynamic ARP Inspection (DAI):** on a managed switch, DAI validates ARP replies against a trusted DHCP-snooping binding table and drops forged ones, preventing the attacker from redirecting the victim’s traffic.
- **Static ARP entries:** pinning the server’s IP-to-MAC mapping on the client (and vice versa) makes the client ignore forged ARP replies. This is impractical at scale but effective in a small, fixed test bed and is easy to demonstrate.

We plan to show that once the MITM is prevented, the attacker loses access to the live sequence number and the attack collapses to the far harder blind case — illustrating that defending the ARP layer and the TCP layer are complementary.

References

- [1] W. Du, “Attacks on the TCP Protocol,” Chapter 16 in *Computer & Internet Security: A Hands-on Approach*, 2nd Edition. Available at <https://www.handsonsecurity.net/>.
- [2] SEED Labs, “TCP Attacks Lab” and “ARP Cache Poisoning Attack Lab,” Syracuse University. <https://seedsecuritylabs.org>.
- [3] A. Ramaiah, R. Stewart, M. Dalal, “Improving TCP’s Robustness to Blind In-Window Attacks,” RFC 5961, IETF, 2010. <https://datatracker.ietf.org/doc/html/rfc5961>.
- [4] J. Touch, “Defending TCP Against Spoofing Attacks,” RFC 4953, IETF, 2007. <https://datatracker.ietf.org/doc/html/rfc4953>.

- [5] P. Watson, "Slipping in the Window: TCP Reset Attacks," Presentation, CanSecWest, 2004.
- [6] N. Weaver, R. Sommer, V. Paxson, "Detecting Forged TCP Reset Packets," Proc. NDSS, 2009.
- [7] M. S. Hossain et al., "TCP Reset Attack on Video Streaming," 2018.
- [8] Tanjim131, "TCP-RST-VideoStreaming-Attack," GitHub repository. <https://github.com/Tanjim131/TCP-RST-VideoStreaming-Attack>.